Scrubgnome Away from Home

kultrshok

Scrubgnome Away from Home

Twig's pocket edition. Strip the snitchy metadata out of phone videos before you post them.

> "Great gears! Your video is blabbing where, when and with what you filmed it. Hand it over."
> — Twig

Scrubgnome Away from Home (AFH) is the Android sidekick to desktop Scrubgnome. It does one job fast:

1. **Pick** a video (or *Share* one to "Scrub with Twig" from your gallery).
2. **See** what it's leaking: GPS location, recording time, phone make/model, software version, hidden data tracks.
3. **Scrub & save** a clean copy to `Movies/Scrubgnome/` with a random filename.
4. **Share / upload** the clean copy.

Optional: **Listen for hidden tracking tones.** Twig scans the audio for inaudible "ultrasonic beacon" signals (17.5–22 kHz) and warns you if he hears any.

Two flavours, same brain: - **Android app** (below): install the APK. - **Web version for iPhone / any browser** (`web/`): open the page in Safari, tap *Share* → *Add to Home Screen*, and it works like an app, even offline. It reads the video locally and never uploads it; the page's security policy forbids it from sending data anywhere.

Privacy promise: the Android app has **no internet permission at all**. Android won't let it talk to the network even if it wanted to. Everything happens on your phone.

Install (no app store needed)

1. Open the repo's **Actions** tab → newest green **Build APK** run → download **ScrubgnomeAFH-apk** (a zip holding the APK). Tagged versions also show up under **Releases**.
2. Copy the APK to your phone and tap it. Android will ask you to allow installs from that app (Files/Chrome). Allow it once.
3. Requires **Android 10 or newer**.

iPhone / web version

1. Open the web app's address in Safari (published from this repo via GitHub Pages once the repo is public; see `.github/workflows/pages.yml`).

2. Tap **Share → Add to Home Screen**. Twig gets his own icon and works offline after the first visit.
3. **Pick a video** → choose **Browse** to pick from Files (that hands over the untouched original; the Photo Library option may give a re-compressed copy).
4. **Scrub & save clean copy** → **Save / share** → **Save Video** puts the clean copy in Photos.

How it stays light on memory: it never loads the whole video. It reads only the small header boxes, then stitches the clean copy together from untouched slices of the original plus tiny patches. The tone check pulls out only the audio frames and lets Safari decode those. The JavaScript scrubber produces byte-for-byte the same output as the Android (Java) one.

What gets removed

Leak	Where it hides	What Twig does
GPS location	`©xyz`, QuickTime location keys, XMP	box neutralized and zeroed
Recording date/time	`mvhd` / `tkhd` / `mdhd` headers, `©day`, XMP	timestamps zeroed
Phone make & model	`com.android.model`, `©mak`/`©mod`, Apple keys	box neutralized and zeroed
OS / software version	`com.android.version`, `©swr`, `©too`	box neutralized and zeroed
Hidden data tracks	Pixel `mett`, iPhone `mebx`, camera-motion/GPS, subtitle tracks	sample bytes zeroed **and** track removed
XMP sidecar blobs	top-level `uuid` boxes	neutralized and zeroed
Vendor trailers	junk after the movie data (e.g. Samsung)	cut off
Handler names	`hdlr` (“VideoHandle”, “Core Media…”)	blanked
Original filename	`PXL_20261108_001736.mp4` *is* a timestamp	clean copy gets `scrubbed_<random>.mp4`

Lossless: nothing is re-encoded. Metadata boxes become same-size `free` boxes full of zeros, so every pointer in the file stays valid and the video and sound are bit-for-bit untouched. After scrubbing, Twig re-scans the result and refuses to save if anything survived.

Supported: MP4, MOV, 3GP (everything phones record). Not supported: WebM/MKV.

The hidden-tone check (and its limits)

Cross-device ad tracking sometimes plays near-ultrasonic beacons (steady pilot tones or short data bursts) that phones can hear and people can’t. Twig decodes the audio on-device, runs an FFT over 17.5–22 kHz and reports:

Result	Meaning
🔴 **Steady inaudible tone**	A constant narrow tone, beacon-like. Could also be electronics (chargers, displays, pest repellers).
🟠 **Possible data bursts**	Narrow blips that come and go or hop frequencies.
🔵 **Some ultrasonic noise**	Broadband hiss, no beacon pattern.
🟢 **No tracking tones** / **No room for hidden tones**	Clean, or the phone's encoder already cut everything above the beacon band.

It's a **warning, not proof**. It can miss a very faint beacon buried under loud full-range sound, and it can't detect watermarks hidden *inside* the audible range. The pocket edition doesn't remove tones; use desktop Scrubgnome's audio cleaning for that.

Build it yourself

Easiest: push to GitHub and let Actions build it (see `.github/workflows/build-apk.yml`).

Android Studio: *File → Open* this folder, wait for Gradle sync, press ▶ with your phone plugged in (USB debugging on).

Desktop self-tests (Android core needs Java 17+, web version needs Node 22+; both need ffmpeg):

```
bash tools/selftest.sh
bash tools/web-selftest.sh
```

Project layout

```
app/src/main/java/com/kultrshok/scrubgnome/
├── core/Mp4Scrubber.java         ← the scrubber (pure Java, no Android; reusable module)
├── core/UltrasonicAnalyzer.java  ← the tone detector (pure Java, no Android; reusable module)
├── AudioToneScanner.java         ← Android audio decoder that feeds the analyzer
└── MainActivity.java             ← the one-screen UI (Twig's voice lives here)
web/
├── index.html                    ← the web/iPhone app (one screen)
├── js/scrubber.js                ← JS port of the scrubber (lazy, streams from the File)
├── js/tone.js                    ← JS port of the tone detector + AAC audio extractor
├── js/app.js                     ← UI glue, Twig's voice
└── sw.js, manifest.webmanifest   ← offline + Add to Home Screen
tools/                            ← self-tests
```

No third-party libraries. The two `core/` classes drop straight into any Java/Kotlin project.

License

MIT © kultrshok. See LICENSE.

Generated with AI